Graph Neural Networks

A practical primer for your GNN assignment — built for someone coming from standard neural nets and predictive coding, new to graphs.

This is a working reference, not a textbook chapter. It's organized around the five steps of your task description (domain/dataset, problem formulation, preprocessing, architecture/training, evaluation) so you can read it once end-to-end, then come back to whichever section matches the step you're stuck on.

> **How to use this document**
>
> Sections 1–3 build the mental model — read these first, in order.
>
> Sections 4–6 are the practical playbook — keep these open while you code.
>
> Section 7 maps everything directly onto your rubric.

1. Why graphs need a different kind of network

A normal feedforward network or CNN expects input of a fixed, regular shape: a vector of features, or a grid of pixels. A graph does not look like that. A graph is a set of **nodes** (entities) connected by **edges** (relationships), and both the number of nodes and the number of neighbors per node can vary from one example to the next. A social network, a molecule, a citation network, a road network — all of these are naturally graphs, not grids or sequences.

The core problem a GNN has to solve is: **how do you run something like a neural network over data with an irregular, variable-size connectivity pattern, while producing the same output no matter how you happen to order or index the nodes?** That second requirement is called **permutation invariance**, and it is the main reason GNNs look different from the MLPs you already know.

Coming from standard NNs, map the concepts like this:

What you already know	GNN equivalent	Why it's different
Fixed-size input vector	Node/edge feature matrix (variable rows)	Number of nodes/edges changes per graph — no fixed input shape.
Layer = one learned transformation	GNN layer = message passing round	A layer updates every node using its neighbors, not just itself.
Forward pass	Stack of message-passing layers	After k layers, a node has 'seen' everything within k hops.
Backprop / gradient descent	Identical — unchanged	Training loop, loss, optimizer all work the same way you already know.
Convolution kernel scanning a fixed grid	Aggregation function over a variable number of neighbors	Sum/mean/max replace the fixed-size convolution kernel.

The reassuring part: everything you know about training neural networks — loss functions, optimizers, overfitting, regularization, train/val/test splits — still applies unchanged. What's new is only *how a single layer*

computes its output.

2. Representing a graph as tensors

Before any model can touch a graph, it has to become tensors. A graph is described with up to four pieces of information:

- **Node features (N)** — a matrix of shape [num_nodes, node_feature_dim]. One feature vector per node (e.g. an atom's element type, a user's profile attributes).
- **Edge features (E)** — a matrix of shape [num_edges, edge_feature_dim], if edges carry their own information (e.g. bond type, interaction strength).
- **Edge index / connectivity** — instead of a dense adjacency matrix (which wastes memory on huge, mostly-empty graphs), frameworks store an **adjacency list**: a [2, num_edges] tensor where each column (i, j) says 'there is an edge from node i to node j'. This is the standard edge_index format in PyTorch Geometric.
- **Global / graph-level attributes (U)** — optional context that applies to the whole graph (e.g. molecule-level metadata), sometimes called a 'master node'.

> **Why not just use an adjacency matrix?**
> An [n_nodes × n_nodes] adjacency matrix is simple but wasteful for sparse, large graphs, and many different-looking adjacency matrices can represent the exact same graph (just with nodes relabeled). A model must give the same answer regardless of node order — that's the permutation-invariance requirement from Section 1. Adjacency lists (edge_index) side-step both problems and are what real GNN libraries use.

3. Message passing: how a GNN layer actually works

This is the one idea that makes everything else in the assignment make sense. A single GNN layer updates every node's embedding using three steps, repeated for every node in the graph:

Step 1 — Gather

For a target node, collect the current embeddings of all its directly connected neighbors (and, if you're using edge features, the edges connecting them too). These are called **messages**.

Step 2 — Aggregate

Combine the variable-length list of neighbor messages into one fixed-size vector using a **permutation-invariant** function — typically **sum**, **mean**, or **max**. This is the step that lets the layer handle nodes with 2 neighbors and nodes with 200 neighbors using the exact same learned weights.

Step 3 — Update

Pass the aggregated neighbor vector (usually concatenated with the node's own previous embedding) through a small learned function — typically a one- or two-layer MLP — to produce the node's new embedding.

> **The key intuition**
> Stack k message-passing layers and, after the k-th layer, every node's embedding has 'absorbed' information from every node within k hops of it — similar to how stacking convolutional layers grows a pixel's receptive field. Stack too many layers, though, and node embeddings start to blur together (over-smoothing) — in practice 2–4 layers is a common, well-justified range for most tasks, and is easy to defend in your report.

Aggregation function cheat-sheet

Function	Good when…	Watch out for
Sum	You care about the total amount of neighborhood 'signal' (e.g. molecule property driven by total bond count).	Sensitive to nodes with unusually many neighbors.
Mean	Neighborhood size varies a lot and you want a normalized, size-independent view.	Can wash out a single strong/outlier neighbor.
Max	You care about the single most salient neighboring feature.	Throws away information from all other neighbors.

4. Matching your task to a prediction type (Step 2 of the assignment)

Once you pick a domain, the assignment asks you to commit to one of four prediction types. The right choice is really about *what you're labeling*:

Task	You're predicting…	Typical example	Output layer applies to…
Node classification	a label for each node	Is this user a bot? Which topic does this paper belong to?	Every node embedding
Edge classification	a label for each existing edge	What type of relationship is this? Is this transaction fraudulent?	Every edge embedding
Link prediction	whether an edge should exist at all	Will these two users become friends? Should we recommend this product?	Pairs of node embeddings (existing + sampled non-edges)
Graph classification	one label for an entire graph	Is this molecule toxic? What does it smell like?	A pooled/aggregated representation of the whole graph

Practical tip: pick the task that matches data you can actually get labels for. Node and graph classification are the most beginner-friendly to implement and evaluate; link prediction requires extra work generating 'negative' (non-existent) edges for training.

5. Preprocessing and splitting graph data (Step 3)

Building the graph

- Decide what a node is and what an edge is *first* — this single decision shapes everything downstream.
- Turn categorical node/edge attributes into numeric features (one-hot encoding or learned embeddings), the same way you would for a normal tabular model.
- Build the edge_index tensor: two parallel arrays of source-node and destination-node indices. For undirected graphs, add both directions (i→j and j→i).
- Normalize/scale continuous node and edge features, exactly as you'd do for any NN input.

Splitting: this is where graphs differ from tabular data

- **Graph classification**: split at the level of whole graphs (e.g. 70% of molecules for training, 15/15 for val/test). This is the closest to a normal train/val/test split.
- **Node classification (single large graph)**: you usually keep the *whole graph* visible to the model (so message passing has real neighbors to use) but only compute loss on a subset of nodes' labels — a **transductive** split using boolean train/val/test masks over the same graph, rather than physically cutting the graph apart.
- **Link prediction**: split the *edges* into train/val/test sets, and for each split generate an equal number of negative (non-existent) edges to classify against — otherwise the model can trivially predict 'yes' to everything.
- In every case, make sure no test-set label leaks into the features or connectivity the model sees during training — state this explicitly in your report, since it's part of the 'correct split strategy' the rubric checks.

6. Choosing an architecture and training it (Step 4)

All GNN layers implement the gather/aggregate/update pattern from Section 3 — they just differ in exactly how the update function works. You don't need to memorize the math to use these; every GNN library ships them as drop-in layers, the same way nn.Linear or nn.Conv2d are drop-in layers.

Layer type	Core idea	Reasonable default when…
GCN (Graph Convolutional Network)	Each node's new embedding is a normalized average of its neighbors' embeddings, passed through a shared linear layer.	You want a simple, fast, well-understood baseline. Start here.
GraphSAGE	Samples a fixed number of neighbors per node and aggregates them — built to scale to large graphs.	Your graph is large, or you want to generalize to unseen nodes at inference time.
GAT (Graph Attention Network)	Learns an attention weight for each neighbor instead of treating them all equally.	Some neighbors clearly matter more than others and you want the model (and you, for interpretation) to see which.
GIN (Graph Isomorphism Network)	Uses sum aggregation plus an MLP, designed to be as theoretically expressive as possible at distinguishing different graph structures.	Graph classification tasks where structural differences between graphs matter a lot.

Design decisions to justify in your report

- **Depth (number of layers)**: 2–4 is standard. More layers = larger receptive field, but risks over-smoothing (node embeddings converging to similar values).
- **Hidden dimension**: 32–128 is a reasonable starting range for small-to-medium datasets; bigger isn't automatically better and studies show smaller models often perform comparably while training faster.
- **Aggregation function**: pick per the cheat-sheet in Section 3, and say why.
- **Readout/pooling for graph-level tasks**: after the last message-passing layer, pool all node embeddings into one vector (sum/mean/max over nodes) before the final classifier — this is the graph-level equivalent of global average pooling in a CNN.

Training loop

This is the part that will feel completely familiar: forward pass → compute loss on the relevant nodes/edges/graphs → backward pass → optimizer step. Use cross-entropy for classification, add dropout and/or weight decay for regularization, and use an Adam optimizer as a sane default. The only GNN-specific addition is deciding your **batching strategy**: for many small graphs (graph classification), you batch multiple graphs together into one big disconnected graph; for one huge graph (node classification), you either train on the full graph each step or sample neighborhoods (as GraphSAGE does) if it doesn't fit in memory.

Recommended tooling

PyTorch Geometric (PyG) is the most widely used library and has built-in GCN, GraphSAGE, GAT, and GIN layers, plus utilities for batching and splitting graphs — it will save you from reimplementing message passing by hand. DGL (Deep Graph Library) is a solid alternative with similar coverage.

7. Evaluating and analyzing your model (Step 5)

Task	Primary metrics	Also worth reporting
Node / edge / graph classification	Accuracy, F1 (macro/micro), confusion matrix	Per-class performance if labels are imbalanced — very common in real graphs.
Link prediction	AUC-ROC, Average Precision (AP)	Precision/recall at a fixed threshold if there's a practical decision to be made.
Regression on graphs	MAE, RMSE, R^2	Residual plots to spot systematic bias.

For the analysis the rubric wants, go beyond the headline number: compare against a simple non-graph baseline (e.g. a plain MLP on node features alone, ignoring edges entirely) to show the graph structure is actually adding value; look at a few misclassified examples and reason about why; and briefly discuss what would happen with a deeper or shallower model, even if you only have time to test one or two variants.

8. Direct map to your rubric

Rubric line (20% each)	Where to look in this document
Domain & Dataset Preprocessing	Sections 2 and 5
GNN Conceptual Understanding	Sections 1 and 3 — be ready to explain message passing, aggregation, and embeddings in your own words
Architecture & Layer Selection	Section 6 — state your depth/hidden-dim/aggregation choices and justify each one
Implementation & Training	Section 6, training loop subsection
Evaluation Metrics & Performance	Section 7

Primary source for the conceptual material above: Sanchez-Lengeling, Reif, Pearce & Wiltschko, "A Gentle Introduction to Graph Neural Networks," Distill, 2021 — https://distill.pub/2021/gnn-intro/. It has interactive diagrams (hover-to-see message passing, an editable GNN playground) that are worth exploring directly once the concepts above feel solid.